DRIFT DETECTOR

Postgres Query Cost Regression Detector

Development Plan & Build Roadmap · Solo Founder Edition

1. North Star & Success Criteria

Before you write a line of code each week, check it against these. If what you're building doesn't directly move one of these needles, stop and ask why you're doing it.

Milestone	Definition of Done
Agent ships data	Binary polls pg_stat_statements and delivers a valid JSON payload to a local endpoint. Zero crashes over 24h.
Drift is detectable	Backend flags a known slow query with z-score ≥ 3 within 15 minutes of regression occurring on test data.
Pilot user installs it	One external user (not you) runs the agent against their real DB with zero hand-holding from you.
First $1 in revenue	One paying customer on the $59/mo Pro plan. This validates the problem is real and people will pay.

2. What You're Building — Component Map

Four components. Build them in this order. Do not jump ahead. Each depends on the previous being stable.

Component	Key File(s)	What it does	Why it matters
1. Collector Agent	*internal/collector/ collector.go*	Polls pg_stat_statements every 5m, fingerprints queries, ships compressed payload	This is the data tap. Nothing works without it. You already have this scaffolded.
2. Backend Ingest	*api/ingest.go store/snapshots.go*	Receives payloads, decompresses, validates, persists snapshots to Postgres	Raw storage layer. No detection until this is solid.
3. Drift Engine	*engine/detector.go engine/baseline.go*	Computes rolling baselines per fingerprint, z-scores, emits anomaly records	The product's core value. Get this wrong and you drown in false positives.

Component	Key File(s)	What it does	Why it matters
4. Event Correlator + UI	*engine/correlator.go ui/timeline.html*	Links anomalies to deploy/migration timestamps, renders timeline view, Slack alert	What turns raw anomalies into actionable root cause. The thing users actually see.

3. Week-by-Week Build Roadmap

7-week timeline. This is aggressive but achievable for a focused solo founder. Each week has one clear output — something that runs, not something that's 'almost done'.

Week	Goal	Tasks	Output
Week 1	**Collector runs locally end-to-end**	• Set up local Postgres with pg_stat_statements enabled • Run go mod tidy and build the scaffolded agent • Point it at local DB, verify JSON payload in stdout • Stub the backend with a simple nc listener or RequestBin • Generate synthetic query load and confirm fingerprints appear	**Running binary**
Week 2	**Backend ingestion endpoint live**	• Scaffold Go HTTP server (net/http or chi router) • POST /api/v1/agent/payload: decompress gzip, validate schema • Write snapshots to Postgres (snapshots table) • Write integration test: agent → backend → DB row visible • Deploy to a cheap VPS (Hetzner CAX11 = €3.29/mo)	**Data in DB**
Week 3	**Drift detection working on real data**	• Implement rolling 14-day baseline per fingerprint (mean, stddev) • Compute z-score on each new snapshot arrival • Write anomaly records when z ≥ 3 AND absolute delta > 50ms • Seed DB with 2 weeks of synthetic data, inject a regression, confirm detection • Add EWMA smoothing to reduce noise on low-call queries	**First anomaly detected**
Week 4	**Timeline UI live**	• Build single HTML page with Chart.js time-series (no frameworks) • Plot mean_time per fingerprint over time • Add vertical lines for anomaly windows • GET /api/v1/projects/{id}/anomalies endpoint feeds the chart	**Visible UI**

Week	Goal	Tasks	Output
		• Show top-3 anomalous fingerprints on load	
Week 5	**GitHub webhook + Slack alert wired**	• POST /api/v1/events/deploy receives GitHub push webhook • Store deploy_timestamp in event_records table • Correlator: join anomalies + events where delta < 6h • Add deploy lines to timeline chart • Send Slack webhook message when anomaly fires	**Alert fires in Slack**
Week 6	**Pilot user running it**	• Write a one-page setup guide (README is not enough) • Build a dead-simple install script: curl | bash • Reach out to 5 target founders — offer free setup call • Shadow at least 1 user during installation • Fix the top 3 issues they hit	**External user live**
Week 7	**Revenue-ready**	• Wire Stripe Checkout for $59/mo Pro plan • Add project-level auth (API tokens per customer) • Implement 7-day data retention for Free tier cutoff • Write a landing page (single HTML file, no CMS) • Post on Hacker News Show HN	**$1 revenue**

4. Daily Development Routine

You are a solo founder. Distraction is your biggest enemy. Here is a non-negotiable daily structure.

Time Block	Activity
First 90 min	Deep work only. No Slack, no email, no Twitter. Ship the current week's primary task.
30 min	GTM work — one outreach message, one post, one forum reply. Consistent, not bursty.
End of day	Write one sentence: what did I ship today? If the answer is 'nothing', something went wrong.

5. Technical Decisions & Rationale

These are locked decisions for the MVP. Do not revisit them before Week 6. Revisiting architecture before you have users is a trap.

Go for the agent binary

Compiles to a single static binary. No runtime. Customer runs it with one command. Any other language requires a runtime installation — that's friction you can't afford when trying to get pilot users.

pg_stat_statements only (no EXPLAIN)

Running EXPLAIN on production queries is dangerous. pg_stat_statements is always-on, read-only, and gives you 90% of what you need. Add on-demand EXPLAIN as a paid feature in v2.

Z-score with absolute threshold

Pure z-score triggers on statistically significant but practically irrelevant changes (e.g. 1ms → 3ms). The absolute threshold (30% AND >50ms) filters those out. This is the difference between a useful product and an alert fatigue machine.

Single Postgres for MVP storage

TimescaleDB or InfluxDB adds operational complexity before you even have a user. Plain Postgres with a timestamp index on snapshot_records is sufficient up to ~50 customers. Add Timescale in v2 if you hit performance issues.

No auth framework — manual tokens

JWT, OAuth, SSO — all overkill. Hard-coded Bearer tokens per project, rotatable via API. Takes 20 lines. Revisit auth only when an enterprise customer requires it.

6. Risks, Blockers & Kill Conditions

As your mentor: most startups don't die from building the wrong thing. They die from refusing to acknowledge it early enough. These are your kill conditions — if you hit one, stop and re-evaluate before burning more months.

KILL CONDITION

If you reach Week 6 and cannot get 3 external users to install the agent — even for free — the GTM thesis is broken. Stop building features and spend 2 full weeks only on user discovery and messaging.

RISK: False positive hell

If pilot users see more than 1 false positive per day in the first week, they will uninstall. Tune thresholds aggressively before pilot. Start with z ≥ 4 and loosen later rather than starting loose.

RISK: Postgres version fragmentation

pg_stat_statements changed column names between Postgres 12 and 13 (total_time → total_exec_time). Your first 3 pilot users will hit this. Handle both versions in the collector before pilot week.

RISK: Scope creep

Your spec already mentions Datadog, Prometheus, Flyway, Liquibase, GitHub, Slack, self-hosting, and enterprise. You are one person. Pick ONE integration per component. Everything else is a distraction until you have 10 paying customers.

MANAGED RISK: Credential security

Agent stores DB credentials in the customer's environment only — this is already correct in the spec. Make sure you say this clearly in the README and landing page. It is a selling point for security-conscious buyers.

7. Go-To-Market — What To Do Right Now

You do not have the luxury of 'build first, market later'. These GTM actions should start in Week 1, running in parallel with development. 30 minutes a day, every day.

Week 1–2: Earlybird conversations

- Post in r/PostgreSQL and r/SaaS: 'Do you track query regressions after deploys? How?' — listen, don't pitch.
- Join Postgres Slack (postgres-slack.com) and Indie Hackers. Find threads about slow queries.
- DM 10 founders of small SaaS products you use. Ask how they handle Postgres performance regressions. Do not mention your product yet.

Week 3–5: Pre-launch waitlist

- Set up a single-page landing page (HTML, no CMS). One headline, one sub-headline, email capture.
- Headline test: 'Find out which deploy slowed your Postgres queries — before your users notice.'
- Write one technical blog post: 'How pg_stat_statements works and why it's underused'. Drives SEO and positions you as credible.

Week 6–7: Pilot and launch

- Show HN: 'Show HN: I built a tool that tells you which deploy caused your Postgres slowdown'
- Post on Indie Hackers and Dev.to with a technical walkthrough of the detection algorithm.
- Offer the first 10 customers 50% off for life in exchange for a public testimonial.

8. Your Next 72 Hours — No Excuses

Stop reading. Do these three things. If you haven't done all three by end of day 3, you're not executing — you're planning.

Day 1	**Run the scaffolded agent against a local Postgres.** Install Postgres locally if you haven't. Enable pg_stat_statements. Build the binary. See it poll and output to stdout. This is non-negotiable — you cannot build a Postgres tool without a working local Postgres.
Day 2	**Stand up the backend ingest endpoint.** Scaffold a minimal Go HTTP server. POST endpoint receives gzip payload from agent, logs the fingerprint count. Data doesn't need to go to DB yet — just prove the end-to-end transport works.
Day 3	**Talk to one potential customer.** One conversation. Not a pitch — a discovery call. Ask: 'How do you currently find out when a deploy slowed down your DB queries?' Record exactly what they say. That language goes into your landing page.

The spec is solid. The code is scaffolded. The only thing standing between you and a working product is execution. Go build.